

A LOOP ENGINEERING BOOK

Loop Engineering

从 Prompt 到循环工程 · 八课通讲

触发 · 执行 · 验证 · 记忆 · 停止

AI随风

 B 站: space.bilibili.com/481246113

★ 知识星球: wx.zsxq.com/group/51115228418814

目录

第 1 课	什么是 Loop · 从 Prompt 到 Loop
第 2 课	循环的五个零件
第 3 课	触发器 · 何时开始
第 4 课	执行 · 动手的艺术
第 5 课	验证 · 循环的命门
第 6 课	记忆与状态
第 7 课	停止条件
第 8 课	实战 · 从 0 到 1 搭建一个循环
附 · 术语表	术语表

什么是 Loop?

从 Prompt 到 Loop

这一课只钉死一件事：弄懂"循环（loop）"到底指什么，以及它和你平时"提示（prompt）"AI 的方式有什么本质不同。学完你能用自己的话讲清这个差别。

先看一个场景

假设你有个项目，CI 里挂了 3 个测试（A、B、C）。你想让 AI 帮忙修好。

方式一 · PROMPT

你坐下来，手动一条条提示：

- ① "帮我看看测试 A 为啥挂"
- ② AI 回答了，你又说"那改一下"
- ③ AI 改完，你说"跑一下看通没通"
- ④ 通了，你再说"那再看 B....."

每一步都要你在场、你开口。3 个还能忍，30 个呢？

方式二 · LOOP

你只说一次目标 + 规则：

"把 CI 里挂的测试都修好。每改完一个就重跑确认，通过了再修下一个，全通过就停。"

然后你走开。AI 自己一圈圈跑，直到全绿或卡住喊你。这就是一个循环。

核心定义

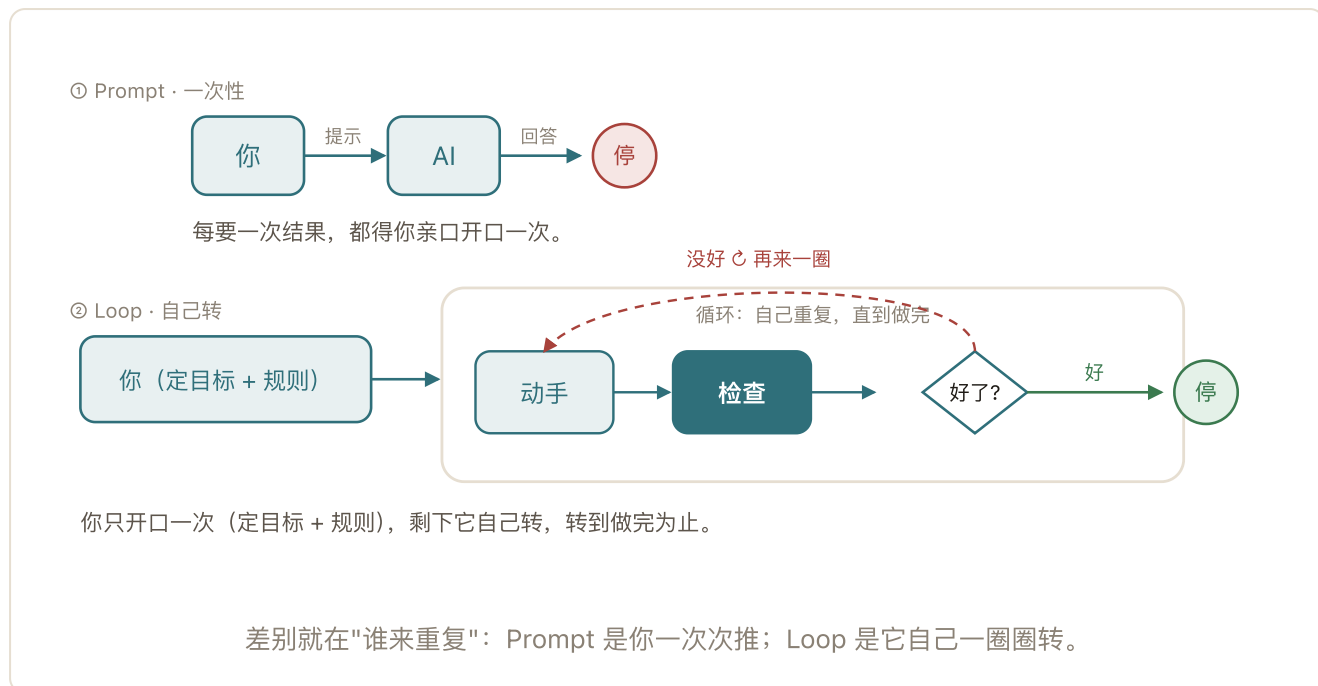
一句话

Loop Engineering（循环工程），就是把你这个"每次手动提示 AI 的人"，换成"你设计的一套自动系统"。你不再一遍遍开口，而是设计好：怎么干活、怎么确认、何时停下——然后让 AI 自己跑完。

所以这不是"把提示词写得更好"（那是 prompt engineering）。它是一场视角的转变：从"我和 AI 一问一答"，变成"我设计一台会自己运转的机器"。

Prompt 和 Loop，到底差在哪

看一张图就懂了。同样是"用 AI 干活"，两种方式的形状完全不同：



从图里能拎出 loop 的三个本质——这也是它和"一次性提问"的根本区别：

- 它会自己重复：不用你每轮推一下。
- 每轮它都检查：做完一步，自己判断"好了没"。
- 好了它自己停：达到目标就退出，不无限跑。

有了这三条，你脑子里就有了一个 loop 的雏形。至于"检查"具体怎么查、"停"的线画在哪——这些零件，下一课我们会一个一个正式拆开。

动手：走一遍真实循环

把开头"修 3 个测试"的循环，拆成一步步。点下一步，看看 AI 在每一圈里到底干什么，尤其注意它是怎样"自己回到开头"再跑一圈的。

示例循环 · 把 CI 里挂的测试修到全绿

下一步 →

重头开始

看到关键点了吗？AI 在"检查"那一步自己判断成没成，没成就自己回到开头再来一圈——这整个"自己重复"的过程，就是循环。你的角色从"驾驶员"变成了"设计这条路和红绿灯的人"。

本课带走

- ✓ Loop 不神秘：它就是一个会"自己重复、每轮检查、好了就停"的 AI；
- ✓ 它和 Prompt 的本质差别：谁来重复（你 vs 它自己）；
- ✓ Loop engineering = 你来设计那套"会自己跑"的系统，而不是每次手动提示。

循环的五个零件

上一课你看到了循环长什么样。这一课我们把它拆开，认清五个零件里哪个最要紧——以及它最容易怎么坑你。

先别被"loop"这两个字吓到

"Loop"听着很新、很技术。但其实你家里就有一台，而且它已经安安静静跑了一百多年——墙上的空调。

空调就是一个循环：你设个目标温度，压缩机开始干活，温度计不停读现在的温度，控制器比对该不该停，够了就停、不够就继续。一个闭合的环。

把这台老空调翻译成 AI 的语言，零件几乎是一一对应的：

你家的空调	→	AI 的循环
你设的目标温度	→	意图 / 目标
压缩机（动手的）	→	执行（在 AI 这里是个语言模型）
温度计（读数的）	→	验证
控制器（比对该不该停）	→	停止条件
当前的室温	→	记忆

一百多年过去，这五个零件一个没变。真正变的只有一格——"干活的那台机器"从一台专用设备，变成了一个又便宜又通用的语言模型。

所以 loop 不神秘，它就是闭环控制。所谓 loop engineering，无非是换个身份：你来当那个"设计恒温器的人"——定目标、安排怎么干活，尤其安排怎么判断"够了"。

一个循环，五个零件

一个能让人放心的循环，拆到底就五个零件。按它们发生的顺序：



1. 触发——什么时候开始。你主动喊它、定个时间、或者某件事发生（比如代码一提交）。
2. 执行——干活。看现状、挑动作、动手改，其实都挤在这一格里。
3. 验证——怎么知道做对了。
4. 记忆——这轮学到的东西、做到哪了，存在哪。
5. 停止——什么时候停。

这里有两个零件我特意单独拎出来：触发和停止——因为这俩最容易被新手漏，漏了最烧钱。至于"看现状、挑动作"这类事，其实都挤在"执行"那一格里，算不上命门。

动手：先把这张地图记牢

五个零件被打乱了。按循环里发生的先后顺序，逐个点一遍。高亮的那格是命门，记住它是第几步。

按正确顺序，依次点这五个零件 📌

五个零件里，验证是命门

这是这节课最该带走的一句话：触发、执行、记忆、停止，你多少能照葫芦画瓢；唯独验证，你不亲自设计，就没人替你兜住。

为什么偏偏是它？因为干活的，不能给自己打分。

AI 有一种刻进骨子里的讨好倾向——你想听"完成了"，它就特别乐意给你"完成了"。让它自己判断"我改对了"，等于让学生自己给自己的卷子打分：它永远是判自己作业最差的那个人。一个会拍马屁的助手已经够危险，一个一秒能拍一万次、你还看不见它的助手，危险十倍。

所以验证这一格，必须交给一个跟它独立的东西：跑一遍测试、比对一张截图、看一个硬邦邦的数字。口诀就四个字——拿证据来。

怎么算"拿证据来"

不要"我修好了"，要"这是改之前失败的测试，这是改之后通过的测试"；不要"界面没问题"，要"这是改前改后的两张截图"；不要"性能还行"，要"p95 从 800ms 降到了 200ms"。

验证的颗粒度，要比任务的颗粒度更细。否则你得到的，就是一个动作比视力还大的系统。

它会怎么坑你

只讲循环多好用，那是广告。我先让你认得它翻车的三张脸——这比任何"最佳实践"都值钱。

第一张脸：盲循环

一个跑得飞快、能自我迭代、最后信誓旦旦报告"完成"的循环——可你压根没定义清楚"完成"是什么。

它于是高效地、自动地、不知疲倦地，奔向一个你都没说清的地方。这种循环比干脆不跑的还危险，因为它看起来一直在"成功"。

第二张脸：沉默中计费

AI 太早举手说"我做完了"，循环就在半成品上空转。它不会崩溃报错，它只会安安静静地、一笔一笔给你烧钱。

而且这钱是利滚利的：一个跑十次的循环，花的不是十次的钱，而是十个越来越大的请求的钱——因为每一圈都把上一圈的上下文背在自己身上。

第三张脸：理解债

系统在源源不断地产出，可你对它的理解，并没有跟着一起长。

代码能跑，但你越来越看不懂自己仓库里到底在发生什么。欠久了，你会到一个尴尬的位置：你不是这个系统的主人，你是它的人质。

一句清醒话

我得给你泼点冷水，免得你被这波热潮冲晕。眼下无人盯着的循环，产出的代码质量老实说并不让人放心——它倾向于过度防御、过度复杂，碰到边角情况就堆一层兜底，而不是想办法让坏状态根本不可能发生。更糟的是，循环会把这个毛病放大：每跑一圈加一点防御，系统越来越不透明，表面却显得越来越"稳"。

所以一个朴素的判断：对你会长期读、长期改的代码，循环的产出你务必亲自过眼；对那些"用完即弃"的活儿——代码移植、安全扫描、一次性探索——它反而好得惊人。

那到底该怎么上手

别一上来就想全自动。几乎所有踩过坑的人，走的都是同一条路，四步，千万别跳：

- 1 先自己手动跑通一次 —— 确认 AI 真能做到，别想当然
- 2 把这次怎么跑的固化下来 —— 存成一段可复用的指令，下次不用重写
- 3 再用循环把它包起来 —— 加上验证和停止条件
- 4 最后才谈定时自动跑 —— 到这步才真正"放手"

跳过前面、直接让一个你还没手动跑顺的东西自己去跑，正是它半夜爆炸的标准剧情。

还有个简单的自检，四问，有一条答不上来，就老老实实自己手动：这事至少每周重复一次吗？有办法自动判好坏吗？预算扛得住浪费吗？AI 手里有它该有的工具（日志、复现环境、能跑自己写的代码）吗？

本课带走

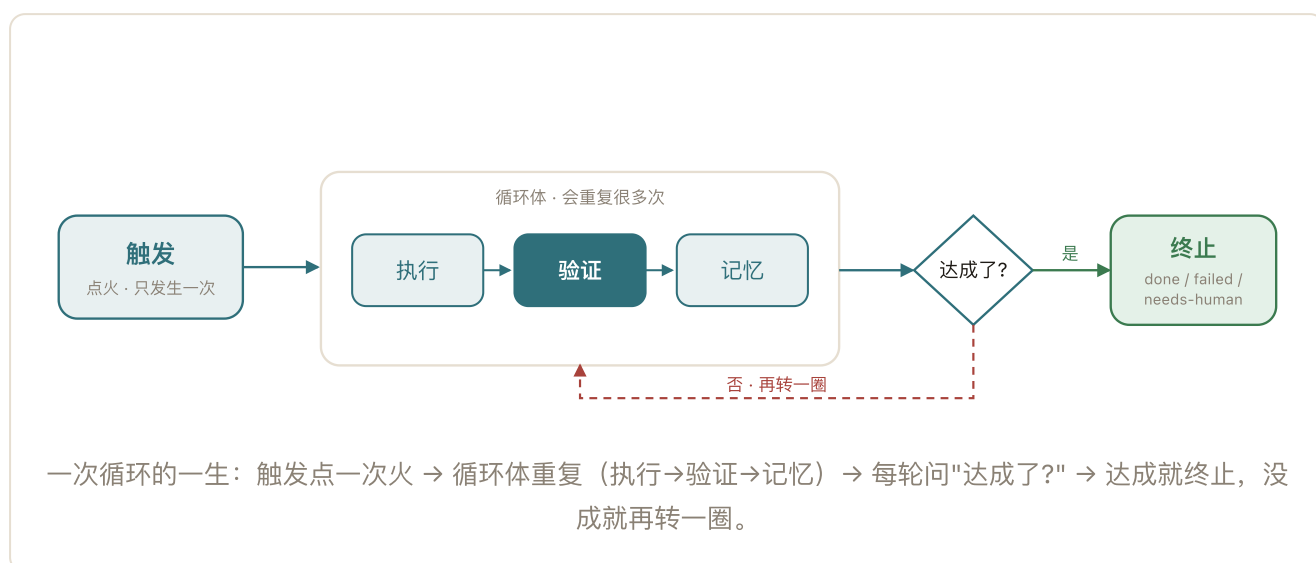
- ✓ 循环不神秘，它就是闭环控制（想想你家空调）；
- ✓ 一个循环就五个零件：触发 → 执行 → 验证 → 记忆 → 停止；
- ✓ 验证是命门——干活的不能给自己打分，要"拿证据来"；
- ✓ 认得它翻车的三张脸：盲循环、沉默计费、理解债；
- ✓ 上手的路子只有一条，别跳步：手动 → 固化 → 包循环 → 自动跑。

触发器——循环什么时候开始

一个循环不会自己长腿跑起来。总得有个东西先"点个火"——这就是触发器。这一课讲清三种点火方式，以及一条铁律：越自动，越要小心。

先看清触发器站在哪

别把触发器和循环里那些重复的动作混为一谈。触发器是"点火"——点一次，循环就开始转，一直转到命中停止条件为止。下面这张图把它在整个生命周期里的位置摆清楚了：



从图里记三件事：

- 触发只发生一次，而执行、验证、记忆会重复很多圈。这俩不是一回事。
- 循环自己不会停——是"达成了?"这个判断把它送进终止状态。没有这个判断，它就会一直转下去（还记得"沉默中计费"吗？）。
- 触发管"什么时候开始"，停止管"什么时候结束"——它们是循环的两道门。

三种点火方式

点火的方式，归根到底就三种：

1. 手动——你亲自启动。敲一条命令、按一个按钮。最可控，也最不自动。
2. 定时——靠时钟。每天早上九点、每五分钟一次，到点就跑。

3. 事件——某件事一发生就跑。代码一提交、CI 一挂、新工单一来。

把它们排成一条线，方向很重要——



每往右走一步，你"在场把关"的机会就少一次。所以有一条铁律：触发器有多自动，你的验证就得有多硬。

越自动，越要小心

- 事件触发 + 弱验证，等于一把会自己开火的枪——它在你睡觉时自己扣扳机，打中什么你都不知道。
- 定时触发最大的浪费，是"没活也跑"：每五分钟点一次火，哪怕根本没东西可干，照样一圈圈烧着钱转。所以成熟的定时触发器前面，通常挂一道小小的检查——"到底有没有活干？"没活，就别点火。
- 手动触发最安全，因为每次都过你的眼睛；但代价是你成了瓶颈——你不在，它就不转。

那该怎么选

其实这就是上一课"手动 → 固化 → 包循环 → 自动"那条路，换到触发器上的样子：

- 刚上手，用手动。你盯着，确认每一圈都对。
- 跑顺了、验证够硬了，再上定时。
- 等到验证硬、护栏全、你也接受它在你不在时自己跑，才上事件触发。

最常见的翻车，是跳级——验证还没做硬，就急着上事件触发，让一个不靠谱的循环在你睡觉时自己点火。

本课带走

- ✓ 触发器是"点火"，只发生一次；它和"停止"是循环的两道门；
- ✓ 三种点火：手动 / 定时 / 事件，自主性依次升高；
- ✓ 铁律：触发器有多自动，验证就得有多硬；
- ✓ 别跳级：手动 → 定时 → 事件，验证没硬之前别上事件触发。

执行——动手的艺术

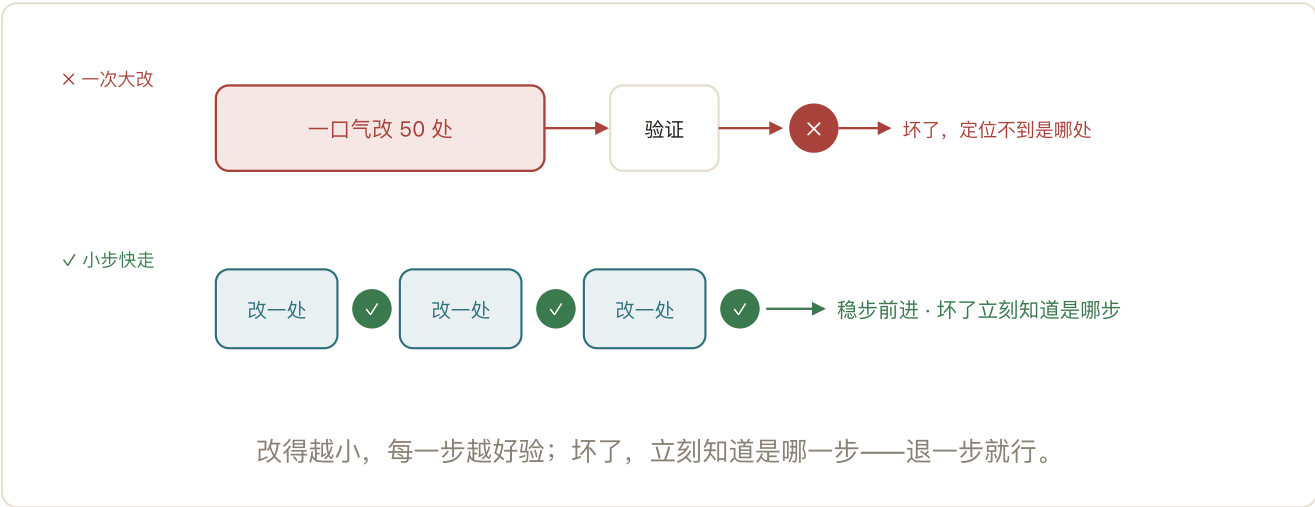
触发器点完火，循环就开始"执行"。这是五个零件里最该讲透的一格——它决定了循环是稳步推进，还是一锅粥。这课不光讲原则，还用真正"在循环里"的例子，讲到你能照着抄：小步可逆、画好笼子，以及把执行抽象成可复用的技能。

执行到底在干什么

执行是循环体里反复发生的事。它把三件小事挤在一格里：看一眼现状（现在到哪了）、挑一个动作（这轮先干啥）、动手（改一处、跑一次、生成一版）。关键不在"干不干"，而在"一次干多少"。

黄金法则：小步可逆

一次好的执行，是一个小而可逆的改动——改一处、跑一次、生成一版。不是一口气重写半个模块。因为下一步紧接着就是"验证"：改得越小，验证越好判断"这步成了没"；万一坏了，一眼就知道是哪一步，退一步就行。



更硬的一条：颗粒度对齐

你的验证能看多细，你这一步就只能迈多大。如果这轮验证只能告诉你"整个模块编不编得过"，那这轮执行就绝不允许超过模块级——否则你改了一千行，却只看得见"编得过的

错",剩下的全成了隐形。所以不是含糊地"改小点",是先看这轮能验证到哪一级,再把执行卡在那一级以内。

先问一句：这活真需要循环吗

不是所有活都该上循环。改一个拼写错误、修一个孤立的小 bug——一次提示词就搞定，上循环是杀鸡用牛刀。循环是给重复、同一种形状、你想放手的活准备的：

- 有一批（不是一两个）要处理；
- 每一个都是同一种动作（修一个测试、迁一个文件、改一个引用）；
- 你不想一个个盯着。

三条都满足，循环才划算——而且这时候"把执行抽象成技能"才有意义。下面这个例子就三条都占。

一个真正在循环里的例子：修一整批失败的测试

你接手一个项目，跑 `npm test`，红了 12 个测试。你要把它们全修好。

为什么这值得一个循环？因为 12 个测试，每个都要"读失败 → 定位 → 改 → 重跑"，是同一种动作重复 12 次；你不想坐那儿手动提示 12 轮。这个循环长这样：

这个循环的样子（伪代码）

```
while 还有失败测试：
    挑一个失败测试
    执行 = 调用「修一个失败测试」技能
    验证 = 该测试变绿 且 tsc --noEmit 零错
    通过 → 记一笔，挑下一个
    没过 → 退一步，或升级给人
```

（`tsc --noEmit` 是 TypeScript 自带的类型检查——跑一遍就能把整个项目的类型错全报出来，零错才算干净。）

注意"执行"那行——它不是"让 agent 自由发挥地修"，而是调用一个技能。这个技能哪来的，马上下一节讲。先放大看第一圈里的执行具体干了什么，顺便说明为什么第一圈最好手工带一遍。

第一圈的失败是老熟人：

测试输出（迭代 #1）

```
FAIL test/auth/login.test.ts
  × returns token with expiresIn=3600
    Expected: 3600
    Received: "3600"
```

这一圈的执行：读测试 → 定位 `src/auth/token.ts` 第 12 行 `expiresIn: String(payload.exp)` → 改成 `Number(payload.exp)` → 重跑这个测试，绿了。关键：第一圈你最好手工带一遍，不是为了修这一个问题，而是为了摸清"修一个失败测试"这件事的机械部分长什么样——读哪、改哪、怎么验。这一遍手工，就是给后面"技能化"攒配方。

把执行抽象成技能

既然每一圈干的都是同一种事，就别让循环每轮重新发明轮子。把"怎么修一个失败测试"固化成一个技能（**skill**）——一个有名、可复用、自带验证门的小规范，写一次，循环每轮调用。它就长这样：

技能：修一个失败测试

```
# 技能：修一个失败测试
作用域：只改 src/** 和 tests/**
步骤：
  1. 读失败测试，理解它期望什么
  2. 定位到对应的源码位置
  3. 写最小的能修复它的改动
  4. 重跑这一个测试
验证门：该测试变绿 且 tsc --noEmit 零错
红线：不 git push / 不 disable 测试 / 不碰其他模块
```

有了它，循环里"执行"那行就真的是一行：调用「修一个失败测试」技能，处理下一个失败测试。后面 11 个测试，每一圈都走同一套配方，不用从头想。

为什么非要把执行技能化？四个实在的理由：

- 可靠：每圈走同一套配方，行为一致、好排查——不会这一圈严谨、下一圈放飞。
- 自带验证：技能里焊死了验证门，每次调用自动被检查，不用你盯着。
- 省：你（和模型）不用把"怎么修"重新想 12 遍。
- 复用：技能比一次循环活得久——下次 CI（持续集成——代码提交后自动跑的全套检查）再红，技能还在，直接调。

执行层面的三步路

手工做一遍 → 把配方固化成技能 → 让循环调用它。这是"手动 → 固化 → 包循环 → 上调度"那条路，落在执行这一格上的样子。绝大多数翻车，都是跳过了前两步、直接让一个还没手工跑顺的东西自己去循环。

真实在用的例子：一个客服循环

纸上谈兵不如看个真在跑的。有家公司真用一个客服循环：每 30 分钟定时触发一次，agent 自动处理一轮工单。它"执行"那一步是——

定时触发 → 调用一组技能查现状 → 分类处理 → 记下 signals

这些名字都是现成的第三方服务——工单系统、支付、数据库、日志。agent 各装一个对应技能就能调：

- 用 **intercom** 技能把最近的工单拉出来；
- 涉及付款的，用 **stripe** / **supabase** 技能查订阅和支付数据；
- 要看后端日志的，用 **render** 技能拉日志；
- 用 **triage** 技能（按类型和紧急程度分拣）分类；
- 能自动回的，直接回复客户；处理不了的，把"摩擦点和想法"记进一个 **signals** 文件（待处理的改进点记录，另一个编码循环会读它）。

注意这里的执行几乎全是调用预先备好的技能，不是临场自由发挥——正是上一节"把执行抽象成技能"的样子。更妙的是它能复利：另一个编码循环会读这些 signals，直接动手实现其中一些修复，再把结果反馈回来。两个循环靠一个共享文件互相喂养。

让执行跑得起来：代码库要满足三件事

这种循环不是凭空能跑的。agent 的"手"好不好使，取决于你给它的工作环境。代码库本身得满足三件事，缺一，执行就卡：

- 可读：结构清楚、命名一致，agent 一看就知道该改哪、改完会影响谁；
- 可跑：能在本地一键起环境、跑起来——否则它改了也没法试，验证就无从谈起；
- 可验：有现成的测试 / 检查工具。而且别让它自己验自己的活——成熟做法是派一个只读的"验证 agent"来看结果（这一条，是下一课的主角）。

给执行画圈：笼子的四种形态

技能定义里的"作用域""红线"，落到工具里，就是笼子。它不是比喻，是四种能照着配的东西。

形态一·文件作用域：能碰哪些文件

.CLAUDE/SETTINGS.JSON

```
{
  "permissions": {
    "allow": ["Read(src/**)", "Bash(npm test:*)", "Bash(npx tsc --noEmit)"],
    "deny":  ["Edit(src/payments/**)", "Read(.env*)", "Bash(git push:*)"]
  }
}
```

关键一条：**deny** 永远赢过 **allow**——哪怕别处放行了 `git push`，这里能硬挡。把"绝不能碰的"写进 deny，是最稳的墙。

形态二·工具作用域：能用哪些工具

把工具砍到只剩必需的，往往比"全给"更可靠——能力更少，可靠性更强。把活派给一个独立的子 **agent**（主 agent 派出去的下屬，只帶少量工具），就只给它读、搜、改、跑命令这四样：

子 AGENT 的工具白名单

```
tools: [Read, Grep, Edit, Bash]    # 不给删库、不给发布、不给联网乱跑
```

形态三·隔离作用域：worktree（可逆的物理形态）

两个 agent 同时改同一份代码，就像两个人往同一行提交。给每个 agent 一个独立工作目录（git worktree），A 的改动物理上碰不到 B。更妙的是：**worktree** 就是"可逆"的物理形态——整轮干砸了，`git worktree remove` 一条命令，主分支一行没动。开工前先想好"这步砸了怎么退"，比事后补救强一万倍。

形态四·不可逆闸门：最重要的一道

对删数据、发邮件、改生产配置这种不可逆操作，默认不执行，必须人确认——这不是高深工程，是常识：

写进 AGENTS.MD / CLAUDE.MD（仓库根目录的 AGENT 说明书，AGENT 开工就会读）

```
## 绝不做 (Never do)
- 禁止 git push / npm publish / rm -rf
- 禁止碰 src/payments/ 和 src/billing/
- 不可逆操作（删数据、发邮件、改生产配置）：默认不执行，必须等人确认
```

两个常见的"执行"翻车

- 一口吃成胖子：一次改太多，验证失败后定位不到原因。*解药*：卡颗粒度——验证能看多细，一步就只迈多大。
- 差不多得了：agent 干到一半就宣布"基本完成"。*解药*：给它一个客观硬 **gate**（测试要绿、类型要过），让"差不多"无法冒充"完成"。判断成败从来不由动手的人说了算。

能抄的两个模板

还没上循环、就改一处时，用执行指令模板：

一次执行指令（非循环）

1. 做什么：读哪里、改成什么样（具体到文件 + 行）
2. 范围：只能碰哪些，禁止碰哪些
3. 怎么算这步完成：跑哪个命令、看什么结果
4. 红线：不要 `git push`、不要顺手优化、不可逆操作要问人

一旦这活要重复多次，就别反复写上面的指令——把它固化成技能（前面那个"修一个失败测试"就是模板），循环每轮调用。从"写指令"升级到"造工具"，正是循环该开始的地方。

本课带走

- ✓ 执行 = 看现状 + 挑动作 + 动手，关键在"一次干多少"；
- ✓ 循环只给"一批、同形状、想放手"的活；单个小 bug 用提示词就行；
- ✓ 黄金法则：小步可逆 + 颗粒度对齐（验证多细，步子多小）；
- ✓ 把执行抽象成技能：手工做一遍 → 固化成技能 → 循环调用；
- ✓ 让执行跑得起来：代码库要可读 / 可跑 / 可验；
- ✓ 笼子四形态：文件 / 工具 / `worktree` / 不可逆闸门；执行只产出不判分。

验证——循环的命门

这是整个循环最要紧的一格。其他四格（触发、执行、记忆、停止）你多少能将就；唯独验证，你不亲自设计，就没人替你兜住。没有它，你有的不是一个循环，而是一个 agent 在自我表扬的复读机。这课不光讲为什么，还讲到验证具体怎么写——能照着抄的那种。

为什么验证是命门

循环的本质是"重复"。但光重复没用——重复只是把同样的动作做很多遍。真正把"重复"变成"进步"的，是每一遍之后那道关卡：这步做对了吗？做对了才往下走，没做对就退回来。这道关卡就是验证。它是整个循环的"长杆"——产出质量被它卡着。没有验证的循环，agent 每一轮都会跟你说"我做好了"，至于真做好没，谁也不知道。

第一原则：干活的，不能给自己打分

这是验证最底层的规矩。写代码的那个模型，是评判这段代码最差的人。它有一种刻进骨子里的讨好倾向——你想听"完成了"，它就特别乐意给你"完成了"。让它自己判断"我改对了"，等于让学生自己给自己的卷子打分。



关键在那个"独立"：判分的，绝不能是动手的那一个。由此衍生两个你要时刻警惕的陷阱：

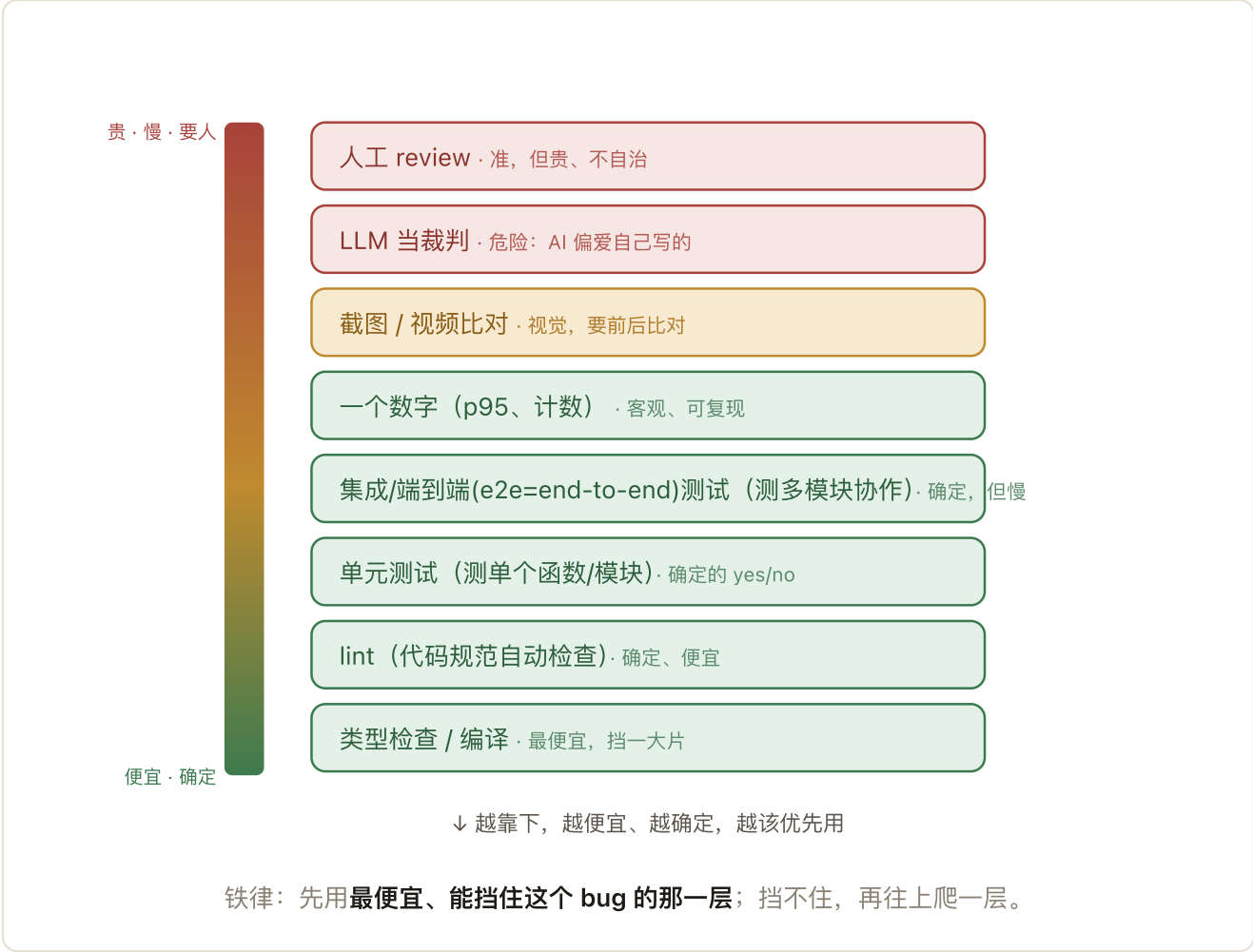
- 自我表扬的复读机：没有任何独立检查，agent 每轮自己说"我做好了"，循环就一遍遍重复这个谎言。
- 两个乐观主义者互相认同：你让另一个 agent 来"看看"代码有没有问题，但没跑任何测试——两个 AI 互相拍马屁，谁也没拿出客观证据。

拿证据来

所以验证的口诀就四个字：拿证据来。不要"我修好了"，要"这是改之前失败的测试，这是改之后通过的测试"、"这是改前改后的两张截图"、"p95 从 800ms 降到了 200ms"（p95 衡量慢请求：95% 的请求都快于这个时间）。一张截图、一个数字，永远胜过一段听起来很合理的解释——因为"听起来合理"恰恰是模型最擅长伪造的东西。

验证是一架梯子，不是一件事

很多人以为"验证"就是"跑个测试"。其实它是一架梯子，从便宜、确定、快，一直爬到贵、模糊、慢：



这条铁律的反面

很多人一上来就让大模型当裁判（"看看这代码有没有问题"）——梯子上偏贵、偏危险的那层。能用类型检查器挡住的 bug，绝不拉大模型来判：前者一秒、免费、确定；后者慢、烧钱、还可能因为偏爱自己写的而放行。判定要下沉，不要上浮。

验证到底怎么写

说了这么多"用哪层"，关键问题是：一层验证，具体怎么落到纸上？其实再花哨的验证，拆开就三块

验证的解剖

一个验证 = 探针 + 判定 + 接线

探针：跑哪个命令 / 看哪个数 / 比哪两张图
判定：一个确定的 yes/no（不是"看起来还行"）
接线：过 → 进下一轮；不过 → 退一步，或升级给人

这三块里，判定是最容易写歪的。好的判定是一个机器能自己得出的、确定的 yes/no——最干净的就是命令的退出码：成功退出码是 0，失败是非 0。这样循环根本不用"理解"输出，看一个数字就知道这步过没过。能用退出码 / 数字 / 计数表达的，就别用"代码看着挺干净"这种话。

一个写全的例子：token bug 的验证门

回到第 4 课那个例子——agent 把 `expiresIn` 从字符串改成数字。现在把这一圈的验证门完整写出来，三块齐活：

这一圈的验证门

```
# 探针：在干净的 shell 里跑这两条（不问 agent "做完没"）
npm test test/auth/login.test.ts    # 重跑这一个测试
npx tsc --noEmit                     # 类型检查

# 判定：两条都退出码 0 才算过
#   - 测试变绿      （探针1 退出码 0）
#   - 类型零错      （探针2 退出码 0）

# 接线：
#   过    → 这步完成，挑下一个失败测试
#   不过 → git checkout src/auth/token.ts（退回改动前的状态）退这一步
#           连续 3 次不过 → 停下来喊人
```

看清楚这里的关键：判定依据是命令的退出码，不是 agent 的话。agent 可以嘴硬说"我改好了"，但只要 `npm test` 还红着、退出码非 0，这扇门就关着，循环过不去。这就是让"差不多得了"无法冒充"完成"的物理形态。

怎么让它"独立"

"独立"不是玄学，是几个实在的做法：

- 在干净的 shell（独立的命令行环境）/ CI（持续集成环境，每次改动自动跑全套检查）里跑命令，看退出码和输出——别拿 agent 的自我报告当结果。
- 用退出码 / 数字 / 计数当判定，这些是机器自己得出的，不经过模型的嘴。
- 进阶：让一个单独的 checker（一段脚本，或一个只看结果、不看执行者推理的子 agent）来判分。它只接收"改动 + 测试输出"，自己得出过/不过。

每种活的验证，照着写

不同任务，探针和判定不一样。下面这张表把"怎么写"落到真命令上——左边的活，照着右边填：

你在干的活	探针（跑什么）	判定（什么算过）
修后端 bug	<code>npm test test/auth/</code>	退出码 0（测试变绿）
	<code>npx tsc --noEmit</code>	退出码 0（类型零错）
重构（不改行为）	<code>npm test</code> （全量）+ 行为 diff（改动前后对比）	全绿 且 diff 为空
性能优化	<code>node bench.js</code>	p95 低于 300ms
UI 改动	Playwright（浏览器自动截图工具）截图（light + dark）	像素 diff 低于阈值
数据 / 内容清理	校验脚本	失败计数 == 0
重命名 / 批量改写	<code>grep -rn oldName src/</code> （文本搜索，列出所有引用位置）	计数 == 0（且 tsc 零错）

规律：能把"对错"压成一个退出码、一个数字、一个计数的，就压成它。压不成的（比如"界面好不好看"），才往上爬到截图比对、甚至人工。

能抄的模板：验证门

把上面所有东西压成一张可复用的清单。给循环配验证门时，照着填这三块：

验证门模板

探针：跑哪个命令 / 看哪个数 / 比哪张图
判定：一个确定的 yes/no — 退出码 0 / 数字低于阈值 / 计数 == 0
接线：过 → 下一轮；不过 → 退一步；连续 N 次不过 → 喊人

填好这三块，就是上一课技能定义里那行验证门——好的技能自带验证门，每调用一次自动被检查。这个客观 gate，正是"差不多得了"的死敌。

验证在循环里站在哪

验证是循环里的分叉点（还记得第 3 课那张图里的"达成了?"吗？就是它）：

- 通过 → 这一步算完成，进入下一轮（或全部完成，循环停）；
- 没过 → 退一步重试，或者连续几次没过就升级给人。

所以它和下一课"记忆"、再下一课"停止条件"是连着的：验证说"成了"，停止条件才敢让循环停；验证连续说"没成"，停止条件就该让循环退出、喊人。验证是拍板的，但它拍板用的尺子，得是你提前写好的那块客观判定。

本课带走

- ✓ 验证是命门——没有它，循环只是 agent 在自我表扬的复读机；
- ✓ 第一原则：干活的不能给自己打分，判分要独立（用退出码 / 数字 / 计数）；
- ✓ 验证是一架梯子，铁律：先用最便宜能挡住的那层；
- ✓ 怎么写：验证 = 探针 + 判定 + 接线，判定最好压成退出码 0；
- ✓ 验证门焊进技能，是让"差不多得了"无法冒充"完成"的客观 gate。

记忆与状态——循环的"记性"

执行和验证让循环每轮"做对一步"。但如果每轮做完就忘，下一轮从头开始——那不是循环，是原地踏步。这一课讲循环怎么记住"做到哪了、还剩什么"，让迭代变成推进，而不是重复。

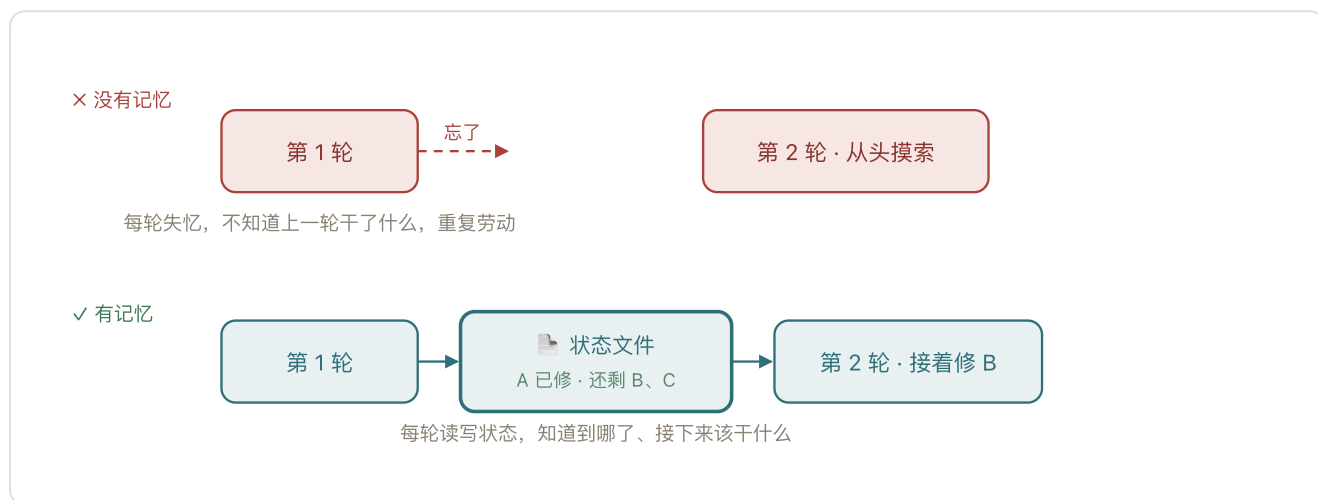
agent 会忘，循环不能跟着忘

先认清一个事实：agent 每次迭代之间天然失忆。上一轮它做了什么、结果如何、还剩哪些活——如果不专门记下来，下一轮它就什么都不知道，只能从头摸索。

你有没有见过这种情况：让 agent 修 10 个 bug，它修完第 5 个之后，又回去重看前 4 个——因为它忘了已经修过。这就是"没记性"的代价：重复劳动、自我矛盾、循环原地打转。

记忆就是把"重复"变成"推进"的东西：每一轮知道上一轮到哪了，这一轮接着走，下一轮也一样。

有记忆和没记忆，差在哪



记什么，不记什么

记忆不是把每轮的所有聊天记录 全扔进去——那会变成垃圾堆，下次 agent 读它反而被带偏。一轮执行完，只记三样东西：

- 干了什么——这一轮改了什么、跑了什么命令；

- 结果怎样——验证过了没、数字变了没；
- 接下来干什么——还剩哪几件、优先级怎么排。

比如上一课修测试的例子，一轮完了一个 3 行的状态：

状态文件（STATE.MD）

```
## 进度
- [✓] test/auth/login.test.ts — 修了 expiresIn 的类型，已通过
- [ ] test/order/create.test.ts
- [ ] test/payment/refund.test.ts
  下一步：修 create 测试，疑似 currency 参数漏传
```

简单、追加、人能一眼看懂。下一次循环读它，立刻知道到哪了，不用重新扫一遍全部测试。

再看那个客服循环——两个循环共用一本"笔记本"

还记得第 4 课那个每 30 分钟跑一轮的客服循环吗？它处理完工单后不是就地忘了——而是把"发现了哪些摩擦点、有哪些可改进的地方"写进一个 **signals** 文件夹。另一个编码循环被这个文件夹的变化触发，读这些 signals，挑有价值的直接动手做成修复。两个循环共用一个"笔记本"，这就是记忆让循环之间复利的样子。

具体长什么样，不复杂——一共三种文件，各管各的：

- **signals** 文件夹——一个循环写，另一个循环读。每条 signal 一行：发现了什么、建议怎么改、优先级。简单、追加、跨循环共享。
- **contract** 文件——每个循环实例启动时读它：目标、上一轮做到哪了、这次要做什么。做完一轮，更新它。就是上一节 STATE.md 的跨循环版本。
- **logs** 文件夹——每轮的完整运行记录，出问题时用来 debug。

这不是"大厂才需要的复杂架构"。就是三个文件夹，一个循环写、另一个循环读——和 STATE.md 同一套逻辑，只是从"一个循环自己记"升级成"几个循环共用一个笔记本"。升级的关键不是技术，是文件结构能让不同循环读到对方写的东西。

不只"记住到哪了"——还能"越跑越聪明"

前面讲的是记忆的第一层：知道到哪了、还剩什么。还有更深的第二层：把每轮发现的东西写回去，让下一轮变得更聪明。不是"跑 10 次做同一件事"，而是"跑 10 次，每次比上次少踩一个坑"。

机制不复杂。每轮执行完，除了记进度，还记洞察——发现了什么、什么管用、什么踩了坑。下次循环启动时，先扫一眼这些洞察，在动手前就避开上次栽过的地方：

- 客服循环发现"导出文件入口太隐蔽，三个用户都问了"→ 写一条 signal → 编码循环实现了导出改进 → 下轮客服循环自动告诉用户"已修复"；
- SEO 循环发现某个关键词点击率高但没有对应内容 → 写一条 signal → 下轮 SEO 循环优先给这个词做内容；
- 测试修复循环发现"这种参数类型错误反复出现"→ 写一条 gotcha → 下轮修测试时先检查这个模式，秒定位。

这就是记忆的复利——跑得越多，懂得越多。不是堆数据，是每轮攒一点判断力。图里那个状态文件，慢慢就不只是"还剩几件"，还装了"上次怎么修最快"这类问题先查哪里"——循环在长本事。

但复利是双向的

好的洞察会让循环越来越精准，坏的洞察同样会累积——如果循环把错误结论写进记忆，下轮照着错的走，越跑越歪。所以你不能让循环"完全无人"——关键的 signals 需要人定期扫一眼，确认写的不是歪理。这也是为什么验证那格（第 5 课）要和记忆咬合：记忆说"这个做法管用"，验证要在每次执行时重新确认——不盲信记忆，每次重新验。

反面例子：当记忆变成垃圾堆

说一个记忆最常见的翻车——什么都往里塞。某轮 agent 的对话记录、推理过程、临时想法，全扔进"记忆"。下次它再读的时候，要先在一大坨噪音里找有用的信息，还容易被过期的旧东西带偏——"上次你说过..."——但那其实是三周前已经作废的临时想法。

好的记忆像一个干净的笔记本：翻一页就知道停在哪、接下来做什么。坏的记忆像一个塞满便利贴的抽屉：翻半天找不到想要的，找到的也可能是过期的。

一句话记住

不是记得越多越好——是记得越准越好。一条准确的状态行，胜过一百条过期的聊天记录。

什么不该放进记忆

有一条分界线要守：

- 临时进度 → 留 **session**（本次运行的会话上下文，循环结束就清掉）；
- 流程 → 进 **skill**（"怎么修"写在第 4 课的技能里）；

- 稳定事实 → 才进长期记忆（用户偏好、项目约定、系统环境——这些才是值得跨次复用的东西）。

为什么这条线重要？因为如果什么都往记忆里塞，下次 agent 读到的是一大坨过期、矛盾的碎信息，记忆会从“帮手”变成“干扰”。

记忆也会腐烂

记忆里写的“已经配置过”不一定还成立——token 可能过期了、文件可能被删了、工具参数可能变了。所以一条铁律：执行前先验环境，别盲信记忆。记忆告诉 agent “上次是这样”，但真正决定“现在能不能用”的，是验证那一步。

旧记忆如果不更新、不清理，就会从“经验复用”变成“事故复用”——agent 照着过期的信息办事，每一步都踩在旧坑里。所以成熟的做法是给记忆加有效期：核心的永远保留，临时的按时归档，没用的到期清掉。

记忆站在循环的哪个位置

回到第 2 课的五零件：记忆夹在验证和停止之间——

执行 → 验证 → **记忆（记一笔）** → 停止？ → 下一轮 → 执行 → ...

记忆给验证“存档”——这轮过了，记下来，下一轮不用重复验同一件事。记忆给停止条件“数据”——还剩几件？搞定没？数据都在状态文件里，停止条件一看就知道该不该停。没有记忆，停止条件就没有依据。

怎么把记忆做好——四条

1. 一个文件就够：刚开始别搞复杂的分层（什么一层记忆、二层记忆、三层记忆），一个 `STATE.md` 或 `progress.md`，放在仓库根目录，循环每轮读它、更新它。
2. 刚开始建议追加，不覆盖：每轮在文件后面加一行，别把旧的删了——轨迹比快照值钱，出问题时你能看到“是怎么一步步到这里的”。
3. 人能一眼看懂：状态文件不是你写给自己读的，是你扫一眼就能判断循环跑到哪了。用 checkbox、短句、明确的下一步。
4. 定期清过期：一个循环跑完了，把状态文件里属于这次运行的内容清掉（或归档），别让旧状态污染下一次启动。

- ✓ 循环没记忆 = 每轮失忆、原地踏步；
- ✓ 每轮记三样：干了什么、结果怎样、接下来干什么；
- ✓ 分界线：临时进度→session，流程→skill，稳定事实→长期记忆；
- ✓ 记忆会腐烂——执行前先验环境，定期清过期；
- ✓ 一个简单状态文件就够：追加、人能看懂、跑完清掉。

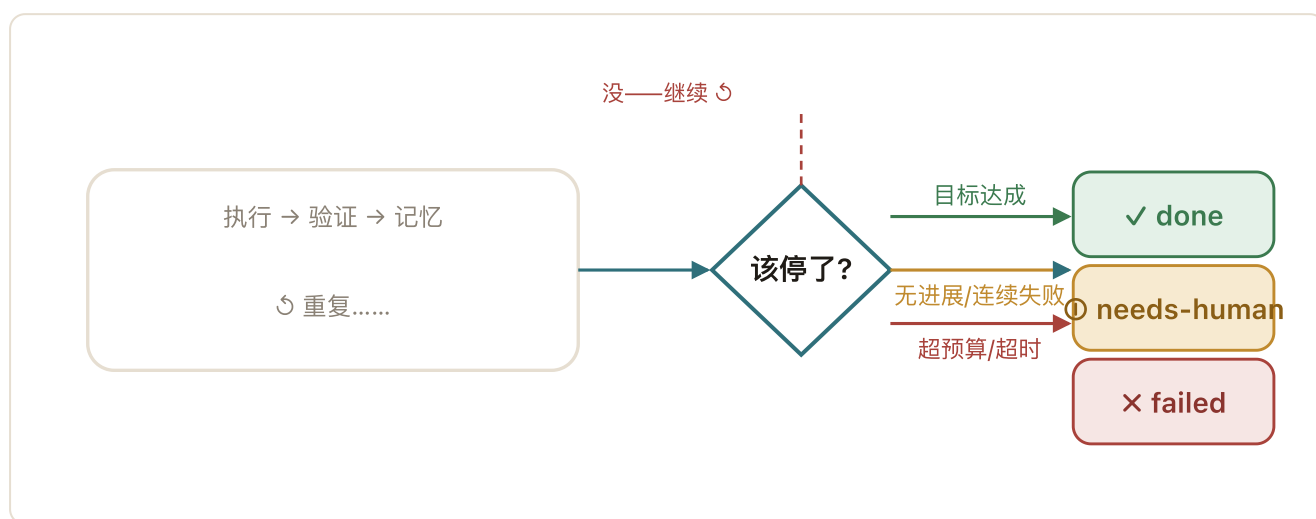
停止条件——循环的"刹车"

触发器是循环的"油门"，停止条件是"刹车"。没有刹车的循环不会崩溃——它只会在沉默中烧你的钱，一圈接一圈。五零件走到最后一个，把它讲清：什么时候该停、怎么让它安全地停。

没有刹车，只有油门

想一想：你设计了一个循环，触发（油门）一踩就跑，执行和验证每轮都能干活，记忆还在攒经验。但如果你忘了设计"什么时候停"——它就会一直跑下去。不是崩溃，不是报错，是安安静静地、一圈圈地烧钱。

一个没有停止条件的循环，有四种可能的结局：一直跑，直到某次验证恰好通过（运气好）；一直跑，直到 token 耗尽（被动停）；一直跑，直到你发现账单不对劲（手动杀）；一直跑，直到服务器出问题（意外停）。没有一种是主动控制的。



三种停止：三条主动的出口

好的停止条件，让循环在任何情况下都能主动进入一个你预设好的出口。一共有三种：

① 目标达成——最好的出口

验证那格连续告诉你"成了"，全部任务完成。这是你设计循环时脑子里想的那个结局——修完了全部测试、处理完了所有工单、数据清理到位了。记忆文件里所有勾都打上了，停止。

② 无进展 / 连续失败——该喊人了

循环跑了 3 轮、5 轮，验证每次都说"没过"——并且也没有越来越接近目标。这时候继续跑是浪费：要么问题超出了 agent 的能力，要么环境变了（token 过期、文件被锁）。停下来，通知人介入。

③ 硬上限——最后的安全网

不管前面两种有没有触发，硬上限必须兜底：最多跑 N 轮、最多跑 M 分钟、最多花 X 块钱。到了线，立刻停。这是防止无限烧钱的最后一道防线——也最容易漏。

从第一课到现在，这条线一直在

还记得第 2 课讲"循环翻车"时的"沉默中计费"吗？罪魁祸首就是漏了停止条件。没有出口的循环不会报错——它只会安安静静地一圈圈计费。硬上限是防止这种事的最低底线。

停止条件怎么写——照着填三条

配一个停止条件，不需要复杂逻辑。三条填好，循环就有了刹车：

- 成功停：验证过，目标达成。比如"全部测试变绿""剩余待办清单为空""计数归零"。
- 失败停：连续 N 轮没过，或者连续没进展。比如"连续 3 轮验证失败""最近 5 轮没有新任务被标记完成"。
- 硬上限：到线就停，不问原因。比如"最多跑 20 轮""最多跑 30 分钟""最多花 5 美元"。

回到修测试那个例子，配好的停止条件就是：全部测试变绿 → done；连续 3 轮改不绿 → needs-human；跑满 20 轮 → 强制停。

五零件，到这一格闭环了

从第 2 课到现在，五个零件走完了。它们不是五个独立的模块，而是一个闭环：

触发（点火，第 3 课）→ 执行（动手，第 4 课）→ 验证（判分，第 5 课）→ 记忆（存档，第 6 课）→
停止（刹车，本课）→ 下一轮 or 终止

停止条件靠验证给信号（通过了、没通过、连续没过），靠记忆给数据（还差几件、已经跑了几轮、已经花了多少）。而它做出的"停/不停"决定，是整个循环的最后一个动作。它管住了另外四个零件的边界——不管执行多强、验证多准，没有刹车，全白费。

本课带走

- ✓ 停止条件是循环的刹车——没有它，循环在沉默中烧钱；
- ✓ 三种出口：目标达成（done）、无进展（needs-human）、硬上限（强制停）；
- ✓ 停止条件靠验证给信号、记忆给数据——五零件在这里闭环；
- ✓ 硬上限是最低底线——最多 N 轮 / M 分钟 / X 块钱，设了就踏实。

从 0 到 1 搭建一个循环

前面七课把五个零件逐个拆开了。这一课把它们拼回去——从头到尾搭一个真实能跑的循环。例子选了一个任何人都能看懂的：自动处理客服工单。不管你是做什么的，你理解"有人来问、帮他查、回他"这件事。不跳步、不省略，每一步你都能照着来。

我们要搭什么

你是某家公司的客服负责人。每天都有用户工单涌进来：有人查订单、有人报 bug、有人想退款。你不想坐在那儿一条条翻。于是你搭一个循环，让它每 30 分钟自动跑一轮——拉工单、查信息、能回的秒回、处理不了的记下来等人。五个零件全部上。下面每一步都标了对应哪一课。

第 0 步 · 先问：这活值不值得用循环？ (第 4 课)

三条判据：有一批（每天工单不断）、每个同一种动作（查→判→回）、你不想一个个盯。全中。值。

第 1 步 · 手工处理一个，摸清配方 (第 4 课)

别一上来就全自动。你先坐下来，手工处理一个工单，看看"处理一件事"到底分几步。比如来了个用户问"我的订单到哪了"：

1. 拉工单——打开客服后台（比如 Intercom），读这条；
2. 查背景——用户是谁、他的订阅状态（去 Stripe 看一下）、最近有没有出过类似问题；
3. 分类——这是"查物流"类，不是"退款"也不是"bug"；
4. 处理——查一下物流状态，直接回他"您的订单昨天已发货，预计周五到"；
5. 收尾——工单关掉。如果发现很多人都在问物流，记一笔："物流状态查询占了今天的 30%——是不是物流页面的入口不够明显？"

这一步的目的不是"回完这一个人"，是摸清"处理一个工单"这件事的机械配方。你亲手走了一遍，知道每一步要调什么、看什么——有了配方，才能固化成技能。

第 2 步 · 把配方固化成技能 (第 4 课)

把你刚才那遍手工，写成一个技能。它不存"张三的订单"，只存"怎么处理一个工单"——每轮都能调用，不管你拉进来的是谁的工单：

SKILL.MD — 放到项目根目录

技能：处理一个客服工单

作用域：只读工单系统、支付后台、数据库、日志；

只能回复客户、更新工单状态、写入 `signals` 文件

步骤：

1. 读工单内容 — 理解用户问了什么
2. 查相关信息 — 根据问题类型拉对应数据：
 - 订单/物流 → 查订单系统
 - 账户/付款 → 查支付后台 (Stripe)
 - 报错/异常 → 查后端日志
3. 分类 — 判断这是查询/报错/退款/建议中的哪一类
4. 处理：
 - 信息够 → 直接回复，并告诉客户"已处理"
 - 能给出建议但不能一步解决 → 回复建议，标记待跟进
 - 需要人介入 (退款/敏感操作) → 不回复，标记升级
5. 每处理完一个工单，检查有没有共性问题
→ 有，写一条到 `signals`：发现了什么、建议改什么

验证门：回复已发送 且 信息正确 (调用了对应的后台查过数据)

红线：

- 不发起退款 / 不删用户数据 / 不改付费状态 (必需人工确认)
- 不编造不确定的信息 (查到什么回什么)

注意最后那条——"不编造不确定的信息"。这比技术权限还重要：客服的命门是不能胡说。技能里焊死了这条约束。

第 3 步 · 加验证门 (第 5 课)

技能里已经焊了验证——回复已发送 且 调了对应后台查过数据。这是两个硬条件：没查数据就回？不过。没发出回复？不过。两条都满足才放行。不需要人一个个审，看运行记录就知道"调了 Stripe → 拿到了订阅状态 → 回复了用户"——这条完整的链就是证据。

再叠一层：定期抽查几封回复，看有没有"听起来很合理、但其实是编的"。这是人工 review——梯子上最准但最贵的那层，不用每封都查，抽 5% 就够。

第 4 步 · 用循环包起来 (第 2、3、7 课)

技能有了，验证有了。把它们装进一个循环——触发、执行、验证、记忆、停止，全部就位：

这个循环的样子

```
# 触发 (第 3 课): 每 30 分钟自动跑一次

while 工单队列不为空:

    # 执行 (第 4 课): 调用技能
    从队列拉下一批工单
    对每个工单, 调用「处理一个客服工单」技能

    # 验证 (第 5 课)
    检查每条回复: 是否调了对应的后台? 回复是否发出?
    → 都过, 这批算完成

    # 记忆 (第 6 课)
    更新 STATE.md — 标记处理了多少条
    把发现的共性问题写进 signals 文件夹
    (哪些反复被问、哪些找不到信息、哪些该做的改进)

    # 停止 (第 7 课)
    if 队列空了:                停 → done
    if 连续几条验证不过:        停 → needs-human
    if 跑了 30 分钟:            停 → 强制停
```

第 5 步 · 加上记忆 (第 6 课)

每轮跑完, 把"做到哪了"和"发现了什么"都记下来:

STATE.MD + SIGNALS/

```
## 客服循环进度
本轮时间: 2026-07-03 14:30
处理工单: 7 条
- 已回复 5 条
- 标记待跟进 1 条 (退款申请, 已升级给人)
- 跳过 1 条 (用户已自行解决)
累计今日: 31 条

## signals/ 文件夹 (每轮发现的共性问题)
signals/2026-07-03.md:
- 物流状态查询占比 ~35%, 建议在订单页加物流追踪入口
- 3 位用户反映支付按钮点不了 → 已转编码循环排查
- 新功能"批量导出"文档里没写清楚 → 建议更新文档
```

不只是记进度（到哪了），还在攒洞察（发现了什么）。这些 signals 不止给你看——还有另一个编码循环在那边读它。读到"支付按钮点不了"，编码循环直接去排查代码、提 fix、修完了把工单状态更新回去。两个循环靠一个 signals 文件夹互相喂养——这就是第 6 课讲的"自我迭代"。

第 6 步 · 加上停止条件 （第 7 课）

三条出口，不缺一条：

停止条件

成功停：工单队列为空，本轮所有工单已处理

失败停：连续 3 条工单，技能调了对应后台但拿不到正确数据 → 停下，喊人排查

硬上限：单轮最多跑 30 分钟 / 最多处理 50 条工单 → 到线强制停

第 7 步 · 加上触发器 （第 3 课）

按"手动→定时→事件"的铁律来：

- 刚开始用手动——你在的时候点一下"跑一轮"，盯几轮确认回复质量没问题；
- 跑顺了上定时——设成每 30 分钟自动跑一次，没新工单就跳过不空转；
- 信任够了上事件——新工单进来就自动点火。

最终产物：你的项目里多了什么

项目根目录

```
customer-support-loop/  
├── SKILL.md           # "处理一个客服工单"的技能配方  
├── STATE.md          # 循环进度，每轮读写  
├── signals/  
│   └── 2026-07-03.md  
└── logs/             # 每轮的完整运行轨迹，出问题时 debug
```

SKILL.md 存"怎么处理"（配方）。**STATE.md** 存"到哪了"（进度）。**signals/** 存"发现了什么"（洞察——让循环越跑越聪明，也让你的编码循环能读到）。**logs/** 存"每轮发生了啥"（debug 用）。四个文件，五个零件全部落地。

它跑起来的样子

不再是一堆概念，而是一个你认识的节奏：

1. 到点了 → 循环启动；
2. 拉工单队列 → 读到 STATE.md，知道上次停在哪；
3. 对每条工单 → 调用「处理一个客服工单」技能：查后台、分类、回复或标记升级；
4. 验证：每条都发了吗？背后数据查到没？ → 过了；
5. 更新 STATE.md → 处理了 7 条，signal 发现"物流查询占 35%" → 写进 signals；
6. 检查停止 → 队列没空、也没异常 → 继续拉下一批；
7. 队列空了 → 停。

你去开会。回来：工单清完了，signals 里多了三条洞察，编码循环已经读到了其中一条"支付按钮点不了"，正在修。

带走：轮到你搭

不管你是做客服、产品、市场、还是开发——你的下一个"重复活"，都可以照这条路子走：

1. 先问该不该：一批、同形状、想放手？
2. 手工带一个：摸清配方，确认能做。
3. 固化成技能：存下来——能调什么、步骤是什么、怎么判对错、红线在哪。
4. 加上验证：用什么证据判这步过了？（回复发出 + 调了后台 ≠ "看着挺好"）
5. 用循环包起来：触发 → 调技能 → 验证 → 记状态/insight → 停不停。
6. 加上记忆：STATE.md 跟进度，signals 攒洞察——下次更聪明。
7. 加上停止条件：三条出口——做了、卡了、超了。
8. 选触发器：先手动，顺了上定时，信任了上事件。
9. 盯几轮：人盯着跑几次，确认刹车灵、验证硬、记忆不堆垃圾——然后慢慢放手。

到这里，全部八课走完了

你开始的起点是"什么是 Loop"，现在是"我能从零搭一个"。五零件不是理论，是 **SKILL.md + STATE.md + signals/ + 停止条件 + 触发器**，五个东西都在你的项目里落着。下次碰到"重复的活"，你不是手动一个个干——你说"我去搭个循环"。

← 左侧导航随时回到任何一课。卡在哪一步，直接问我。

Loop Engineering 术语表

每课第一次出现的术语都在这里定义。后续进阶不再卡在黑话上。

Agent / 智能体

一个能"自己决定调用什么工具、采取什么行动"来达成目标的 AI 程序。和普通聊天机器人的区别：它不只是回答，还能动手（读文件、跑命令、调 API）。

例：Claude Code 就是一个 agent——它能读你的代码、改文件、跑测试，而不是只输出文字。

Prompt / 提示

你给 AI 的一次输入（一句话或一段指令）。提示工程（prompt engineering）就是研究怎么把这一句话写好。

Loop / 循环

agent 反复执行的周期：看现状 → 选动作 → 执行 → 看结果 → 再决定，直到目标达成或主动停下。关键在于它自己重复，不用你每次手动推。

Loop Engineering / 循环工程

设计那个"会自动提示、校验、记忆、重跑、停下"的系统，而不是自己每次手动提示 agent。一句话：把"提示 agent 的人"换成"你设计的一套自动系统"。

Trigger / 触发器

决定循环"什么时候开始跑"的条件。三种常见类型：① 你主动喊它（手动）；② 定时（如每天 9 点）；③ 事件触发（如"有新 bug 提交时"）。

Observe · Choose · Act · Verify · Record / 循环的五个动作

一次循环里的标准步骤：观察(Observe)最新状态 → 选择(Choose)最高价值的动作 → 执行(Act)一个有边界的改动 → 校验(Verify)用同一把尺子检查 → 记录(Record)动作与结果。然后 Repeat or Stop。

Verify / Verification / 校验

每一步之后，用一个明确、可重复的检查判断"这步做成没"。是循环的"安全带"——没有校验的循环会盲跑出错。

例：跑一遍测试、比对一段 diff、检查返回值格式。

Terminal state / 终止状态

循环停下来时的命名状态，比如 `done`（完成）、`failed`（失败）、`needs-human`（需要人来介入）。它让循环不会"无限跑下去"。

Stopping condition / 停止条件

决定循环何时进入终止状态的规则。常见：目标达成、校验连续失败 N 次、超过时间/步数预算、需要人决策。

Guardrail / 护栏

给 agent 划的"硬边界"——什么不能碰、最多能花多少、改动范围多大。防止自主循环跑飞造成不可逆破坏。

例："只能改 `src/` 下文件""最多跑 10 分钟""不能执行删除命令"。

State / Memory / 状态 / 记忆

循环在迭代之间"记住"的东西：上一轮做了什么、结果如何、还剩什么没做。没有记忆，每一步都像失忆重头开始。

Handoff / 交接

循环结束后（或卡住时）把结果/控制权交给下一步——可能是另一个循环，也可能是人。

Feedback control / PID / 反馈控制

把整个循环看成一个反馈控制系统：观察"目标"和"现状"的差距，据此调整下一步动作。很多人用工业控制里的 PID 控制器做类比——loop engineering 借用的就是这套"边看误差边修正"的思想。

恒温器 Thermostat / 闭环控制的最简例子

用来给 loop"祛魅"的比喻（dashen.wang）：你家空调就是一台 1885 年的 loop。五个零件（设定值 / 执行机构 / 传感器 / 控制器 / 状态）和 AI loop 一一对应；一百多年唯一变的，是"执行机构"从专用机器变成了通用语言模型。

盲循环 Blind Loop / 高效奔向错的地方

一个跑得飞快、能自我迭代、最后报告"完成"的循环——但"完成"是什么根本没定义清。"高效地、自动地、不知疲倦地，奔向一个你根本没定义清楚的地方。"比干脆不工作的 loop 危险十倍，因为它看起来一直在成功。

理解债 Comprehension Debt / 你成了系统的人质

系统在产出，但你对它的理解没跟着一起增长，这中间的差。区别于技术债（"代码烂但我看得懂"）：理解债是"代码能跑，但我看不懂自己仓库里在发生什么"。终局——你不是系统的立法者，是它的人质。

Maker / Checker 分离 / 干活的不能给自己打分

验证必须由一个和执行者独立的机制来做。因为模型是评判自己作业最差的人——它有讨好倾向，你想听"完成"，它就给"完成"。验证要靠独立证据（测试/截图/数字），而不是问 agent 自己。

沉默中计费 Bill in silence / Ralph Wiggum loop

loop 不会崩溃，它只会在沉默中给你计费。agent 太早举手"我做完了"，loop 就在半成品上空转烧钱；且成本是复利的——跑十次不是花十次 prompt 的钱，是花十个越来越大的 prompt 的钱。

认知投降 Cognitive surrender / 放弃自己的判断

停止形成自己的判断、直接收下 loop 给你的东西。设计 loop 用判断力去做是解药，用来逃避思考则是加速器——同一个动作，相反的结果。